AI Operating System Workshop

Day 2: Knowledge Graph & Advanced RAG

X-Company
2026

Section 1: Knowledge Graph Fundamentals

A Knowledge Graph (KG) is a structured representation of real-world entities and their relationships. It organizes information as a network of nodes and edges, enabling complex multi-hop reasoning that vector databases cannot perform.

- Nodes: Entities — Person, Organization, Product, Concept
- Edges: Relationships — 'works_at', 'part_of', 'related_to'
- Properties: Attributes on nodes/edges — name, date, confidence_score

KG vs Relational DB vs Vector DB

Feature	Knowledge Graph	Relational DB	Vector DB
Data model	Graph (nodes/edges)	Tables/rows	High-dim vectors
Query type	Traversal, pattern match	SQL joins	Similarity search
Multi-hop	Native, fast	Complex JOINs	Not supported
Schema	Flexible (schema-less)	Rigid schema	No schema
Best for	Relationships, reasoning	Transactions, CRUD	Semantic search
Example	Neo4j, Amazon Neptune	PostgreSQL, MySQL	Qdrant, Pinecone

Section 2: Entity Extraction

NER with spaCy & Stanza

```
import spacy

nlp = spacy.load('en_core_web_sm')
doc = nlp('Apple CEO Tim Cook announced new products at WWDC in San Francisco.')

for ent in doc.ents:
    print(f'{ent.text:20} | {ent.label_:10} | {spacy.explain(ent.label_)}')

# Output:
# Apple                | ORG        | Companies, agencies
# Tim Cook             | PERSON     | People, including fictional
# WWDC                 | EVENT      | Named hurricanes, battles
# San Francisco        | GPE        | Countries, cities, states
```

LLM-Based Entity Extraction

Use LLMs with structured output prompts for more flexible, context-aware extraction:

```
import ollama, json

prompt = '''
Extract entities from the text. Return JSON format:
{"entities": [{"name": str, "type": str, "description": str}],
 "relations": [{"source": str, "relation": str, "target": str}]}

Text: {text}
'''

response = ollama.chat(
    model='llama3.2',
    messages=[{'role': 'user', 'content': prompt.format(text=doc_text)}],
```

```
    format='json'
)
data = json.loads(response['message']['content'])
```

Entity Types Reference

Entity Type	Examples	Extraction Method
PERSON	Tim Cook, Elon Musk	NER / LLM
ORG	Apple, X-Company, OpenAI	NER / LLM
LOCATION	Bangkok, Silicon Valley	NER / LLM
PRODUCT	iPhone, ChatGPT, Qdrant	LLM (context-aware)
CONCEPT	Machine Learning, RAG	LLM (domain-specific)
EVENT	WWDC, CES, conference	NER / LLM

Section 3: Graph Construction Pipeline

Building a knowledge graph involves a multi-step pipeline:

- Step 1: Document ingestion and chunking
- Step 2: Entity extraction from each chunk
- Step 3: Relation extraction between entities
- Step 4: Entity resolution (deduplication)
- Step 5: Graph storage in Neo4j

```
# Step 1-2: Extract entities from documents
entities = []
relations = []
for chunk in chunks:
    result = extract_entities_llm(chunk)  # uses LLM
    entities.extend(result['entities'])
    relations.extend(result['relations'])

# Step 4: Entity resolution
from collections import defaultdict
entity_map = defaultdict(list)
for e in entities:
    entity_map[e['name'].lower()].append(e)

# Step 5: Store in Neo4j
from neo4j import GraphDatabase
driver = GraphDatabase.driver('bolt://localhost:7687',
                              auth=('neo4j', 'password'))
with driver.session() as session:
    for e in entities:
        session.run('MERGE (n:{type} {{name: $name}})',
                    type=e['type'], name=e['name'])
    for r in relations:
        session.run(
            'MATCH (a {{name: $src}}), (b {{name: $tgt}})'
            'MERGE (a)-[:{rel}]->(b)',
            src=r['source'], tgt=r['target'], rel=r['relation'].upper())
```

Section 4: Neo4j & Cypher

Neo4j Setup

```
# Start Neo4j with Docker
docker run -d --name neo4j \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/password \
  -v $(pwd)/neo4j_data:/data \
  neo4j:5.13
```

Cypher Query Language Basics

```
// CREATE node
CREATE (p:Person {name: 'Alice', role: 'Engineer'})

// MATCH and RETURN
MATCH (p:Person) WHERE p.name = 'Alice' RETURN p

// CREATE relationship
MATCH (a:Person {name: 'Alice'}), (b:Company {name: 'X-Company'})
CREATE (a)-[:WORKS_AT {since: 2020}]->(b)

// Traverse 2 hops
MATCH (p:Person)-[:WORKS_AT]->(c:Company)-[:USES]->(t:Technology)
RETURN p.name, c.name, t.name

// Shortest path
MATCH path = shortestPath((a:Person {name:'Alice'})-[*]-(b:Person {name:'Bob'}))
RETURN path
```

10 Example Cypher Queries

#	Use Case	Query
1	Find all persons	MATCH (p:Person) RETURN p.name
2	Find employees of org	MATCH (p)-[:WORKS_AT]->(c:Company {name:'X-Company'}) RETURN p
3	Count nodes by type	MATCH (n) RETURN labels(n), count(n)
4	Find 2-hop connections	MATCH (a)-[*2]-(b) WHERE a.name='Alice' RETURN b
5	Find common colleagues	MATCH (a)-[:WORKS_AT]->(c)<-[:WORKS_AT]-(b) RETURN a,b,c
6	Delete node	MATCH (n:Person {name:'Alice'}) DETACH DELETE n
7	Update property	MATCH (n {name:'Alice'}) SET n.role='Manager' RETURN n
8	Find isolated nodes	MATCH (n) WHERE NOT (n)--() RETURN n
9	Aggregate relations	MATCH ()-[r]->() RETURN type(r), count(r) ORDER BY count(r) DESC
10	Path between nodes	MATCH p=shortestPath((a {name:'Alice'})-[*]-(b {name:'Bob'})) RETURN p

Section 5: GraphRAG

GraphRAG combines vector similarity search with graph traversal to answer complex questions requiring multi-document, multi-hop reasoning.

Architecture:

- Query → Embed → Vector search (retrieve relevant nodes by embedding)
- Retrieved nodes → Graph traversal (expand context via relationships)
- Expanded context → LLM → Answer

GraphRAG-rs Configuration

```
# graphrag-rs config.toml
[storage]
  [storage.neo4j]
  uri = 'bolt://localhost:7687'
  username = 'neo4j'
  password = 'password'

  [storage.qdrant]
  url = 'http://localhost:6333'
  collection = 'workshop_docs'

[embedding]
  model = 'bge-m3'
  dimensions = 1024

[retrieval]
  top_k = 5
  graph_hops = 2
  community_level = 2
```

Section 6: Advanced Retrieval

Multi-Hop Reasoning

Graph-based reasoning allows answering questions that require connecting information across multiple documents:

Example: "Which team members worked on projects that used Python and were delivered in Q4?"

- Step 1: Find all projects delivered in Q4 (graph query)
- Step 2: Filter those using Python (graph property)
- Step 3: Find team members linked to those projects

Community Detection

Identifies clusters of related entities in the knowledge graph. Enables topic-level summarization.

```
# Community detection with NetworkX
import networkx as nx
from networkx.algorithms.community import greedy_modularity_communities

G = nx.Graph()
# Add edges from Neo4j
for rel in relations:
    G.add_edge(rel['source'], rel['target'])
```

```
communities = list(greedy_modularity_communities(G))
print(f'Found {len(communities)} communities')
```

Graph-Enhanced Context

Enrich RAG context with graph neighborhood information:

```
def graph_enhanced_retrieval(query: str) -> str:
    # 1. Vector search
    vector_chunks = vector_search(query, top_k=3)

    # 2. Extract entities from query
    entities = extract_entities_llm(query)

    # 3. Graph expansion
    with driver.session() as session:
        for entity in entities:
            result = session.run(
                'MATCH (n {name:$name})-[r*1..2]-(m) RETURN m.name, type(r[-1])
LIMIT 10',
                name=entity['name']
            )
            # Add graph context to prompt
    return combined_context
```

Section 7: Exercises & Reference

Exercise 1: Build a Company Knowledge Graph

Task: Extract entities from 10 company documents and build a Neo4j graph.

- Include: Person, Organization, Project, Technology node types
- Hint: Use LLM extraction with JSON output format

Exercise 2: Cypher Query Challenge

Task: Write Cypher queries to answer: Who are the top 3 most connected employees? Which technologies appear in the most projects?

- Hint: Use degree() function or count relationships in MATCH

Exercise 3: GraphRAG Pipeline

Task: Implement a GraphRAG pipeline that answers multi-hop questions using your knowledge graph.

- Hint: Combine Qdrant search with Neo4j 2-hop expansion

Reference

- Neo4j Documentation: https://neo4j.com/docs
- GraphRAG paper: https://arxiv.org/abs/2404.16130
- spaCy NER: https://spacy.io/usage/linguistic-features#named-entities
- NetworkX: https://networkx.org/documentation